API Specifications

Revision History

- Sept 1 - Changed DTLearner "Comments" to properly reflect splitting.
- Sept 18 - Updated the BagLearner API specification to clarify that kwargs does not require default arguments.
- Nov 14 - Updated a comment in the P8 API specification so that the comment sd/ed dates match what is in the function signature.

All Projects

- All provided APIs must be used as given below. This means that their function or method signatures (i.e., name, parameters - with or without default arguments, number and order or parameters) or return types cannot be altered.
- You may create additional functions or methods as necessary to support your solution, with the provision that any such functions or methods will not be called directly by the pre-validation and/or private grading scripts.
- A best practice is to include an `author()` function in each `.py` file (except `Insanelearner.py` in Project 3) and an `author()` method within each class (including `InsaneLearner.py` in Project 3).

Project 1: Martingale

`martingale.py`

```
author()

    Returns
        The GT username of the student

    Return type
        str

study_group()

    Returns
        A comma separated string of GT_Name of each member of your study group
        # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

    Return type
        str
```

```
get_spin_result(win_prob)

    Given a win probability between 0 and 1, the function returns whether the probability will result in a win.

    Parameters
        win_prob (float) – The probability of winning

    Returns
        The result of the spin.

    Return type
        bool
```

```
gtid()

    Returns
        The GT ID of the student

    Return type
        int
```

```
test_code()

    Method to test and execute your code
```

Project 2: Optimize Something

`optimization.py`

```
optimize_portfolio(sd=datetime.datetime(2008, 1, 1, 0, 0), ed=datetime.datetime(2009, 1, 1, 0, 0), syms=['GOOG', 'AAPL', 'GLD', 'XOM'], gen_plot=Fal
se)

    This function should find the optimal allocations for a given set of stocks. You should optimize for maximum Sharpe
    Ratio. The function should accept as input a list of symbols as well as start and end dates and return a list of
    floats (as a one-dimensional NumPy array) that represent the allocations to each of the equities. You can take
    advantage of routines developed in the optional assess portfolio project to compute daily portfolio value and
    statistics.

    Parameters
        sd (datetime) – A datetime object that represents the start date, defaults to 1/1/2008
        ed (datetime) – A datetime object that represents the end date, defaults to 1/1/2009
        syms (list) – A list of symbols that make up the portfolio (note that your code should support any
                      symbol in the data directory)
        gen_plot (bool) – If True, create a plot named Figure1.png (or alternatively, plot.png).
```

```
                        The autograder will always call your code with gen_plot = False.

    Returns
        A tuple containing the portfolio allocations, cumulative return, average daily returns,
        standard deviation of daily returns, and Sharpe ratio

    Return type
        tuple

test_code()

    This function WILL NOT be called by the auto grader.


author()
    Returns
        The GT username of the student

    Return type
        str

study_group()

    Returns
        A comma separated string of GT_Name of each member of your study group
        # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

    Return type
        str
```

Project 3: Assess Learners

`LinRegLearner.py`

```
class LinRegLearner.LinRegLearner(verbose=False)

    This is a Linear Regression Learner. It is implemented correctly.
    Students will not implement this leaner. It is provided for reference only.

    Parameters
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

`DTLearner.py`

```
class DTLearner.DTLearner(leaf_size=1, verbose=False)

    This is a Decision Tree Learner (DTLearner). You will need to properly implement this class as necessary.

    Parameters
        leaf_size (int)  - Is the number of samples required for a potential split
        verbose (bool)   - If “verbose” is True, your code can print out information for debugging.
                           If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    author()

        Returns
            The GT username of the student

        Return type
            str

     study_group()
```

```
        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str

    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

RTLearner.py

```
class RTLearner.RTLearner(leaf_size=1, verbose=False)

    This is a Random Tree Learner (RTLearner). You will need to properly implement this class as necessary.

    Parameters
        leaf_size (int)  - Is the number of samples required for a potential split
        verbose (bool)   - If “verbose” is True, your code can print out information for debugging.
                           If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    author()

        Returns
            The GT username of the student

        Return type
            str

     study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str

    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

BagLearner.py

```
class BagLearner.BagLearner(learner, kwargs = {"argument1":1, "argument2":2}, bags = 20, boost = False, verbose = False)

    This is a Bootstrap Aggregation Learner (BagLearner). You will need to properly implement this class as necessary.

    Parameters
        learner (learner) - Points to any arbitrary learner class that will be used in the BagLearner.
        kwargs            - Keyword arguments that are passed on to the learner’s constructor and they can vary according to the learner
                          - NOTE: kwargs is shown in the API as it would be used, but it is NOT required to be a default argument in the same
                            way as bags, boost, or verbose.
        bags (int)        - The number of learners you should train using Bootstrap Aggregation.
                            If boost is true, then you should implement boosting (optional implementation).
        verbose (bool)    - If “verbose” is True, your code can print out information for debugging.
                            If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    author()

        Returns
            The GT username of the student
```

```
    Return type
        str

     study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str


    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

`InsaneLearner.py`

```
class InsaneLearner.InsaneLearner(verbose=False)

         This is a Insane Learner (InsaneLearner). You will need to properly implement this class as necessary.

    Parameters
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    author()                                        # This method cannot be inherited.

        Returns
            The GT username of the student

        Return type
            str


    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

Project 4: Defeat Learners

`DTLearner.py`

```
class DTLearner.DTLearner(leaf_size=1, verbose=False)

    This is a Decision Tree Learner (DTLearner). You will need to properly implement this class as necessary.
    You may revise (if necessary) and use your DTLearner from Project 3 for local testing and use in Gradescope TESTING

    IMPORTANT: We will use our "reference" DTLearner when the code is evaluated for grading in Gradescope SUBMISSION.

    Parameters
        leaf_size (int)  - Is the number of samples required for a potential split
        verbose (bool)   - If “verbose” is True, your code can print out information for debugging.
                           If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data


    author()

        Returns
            The GT username of the student

        Return type
```

```
            str

     study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str


    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) – A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

gen_data.py

```
author()
    Returns
        The GT username of the student

    Return type
        str

study_group()

    Returns
        A comma separated string of GT_Name of each member of your study group
        # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

    Return type
        str

best_4_dt(seed=1489683273)

    Returns data that performs significantly better with DTLearner than LinRegLearner.
    The data set should include from 2 to 10 columns in X, and one column in Y.
    The data should contain from 10 (minimum) to 1000 (maximum) rows.

    Parameters
        seed (int) – The random seed for your data generation.

    Returns
        Returns x, y - data that performs significantly better with DTLearner than LinRegLearner.
                   x – A matrix of feature values
                   y – A vector of prediction or target values

    Return type
        numpy.ndarray


best_4_lin_reg(seed=1489683273)

    Returns data that performs significantly better with LinRegLearner than DTLearner.
    The data set should include from 2 to 10 columns in X, and one column in Y.
    The data should contain from 10 (minimum) to 1000 (maximum) rows.

    Parameters
        seed (int) – The random seed for your data generation.

    Returns
        Returns x, y - data that performs significantly better with LinRegLearner than DTLearner.
                   x – A matrix of feature values
                   y – A vector of prection or target values

    Return type
        numpy.ndarray
```

LingRegLearner.py

```
class LinRegLearner.LinRegLearner(verbose=False)

    This is a Linear Regression Learner. It is implemented correctly.
    Our friendly, working, correct, linear regression learner. It is used by the grading script.
    Do not rely on local changes you make to this file, as you may only submit gen_data.py.

    NOTE: DO NOT MODIFY THIS FILE. IT IS PROVIDED FOR REFERENCE ONLY. YOU WILL NOT SUBMIT THIS FILE FOR YOUR ASSIGNMENT.

    Parameters
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.

    add_evidence(data_x, data_y)

        Add training data to learner

        Parameters
            data_x (numpy.ndarray) – A set of feature values used to train the learner
            data_y (numpy.ndarray) – The value we are attempting to predict given the X data
```

```
    author()
        Returns
            The GT username of the student

        Return type
            str

    study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str

    query(points)

        Estimate a set of test points given the model we built.

        Parameters
            points (numpy.ndarray) - A numpy array with each row corresponding to a specific query.

        Returns
            The predicted result of the input data according to the trained model

        Return type
            numpy.ndarray
```

`testbest4.py`

```
compare_os_rmse(learner1, learner2, x, y)

    Compares the out-of-sample root mean squared error of your LinRegLearner and DTLearner.

    NOTE: DO NOT MODIFY THIS FILE. IT IS PROVIDED FOR REFERENCE ONLY. YOU WILL NOT SUBMIT THIS FILE FOR YOUR ASSIGNMENT.

    Parameters
        learner1 (class:'LinRegLearner.LinRegLearner') - An instance of LinRegLearner
        learner2 (class:'DTLearner.DTLearner') - An instance of DTLearner
        x (numpy.ndarray) - X data generated from either gen_data.best_4_dt or gen_data.best_4_lin_reg
        y (numpy.ndarray) - Y data generated from either gen_data.best_4_dt or gen_data.best_4_lin_reg

    Returns
        The root mean squared error of each learner

    Return type
        tuple

test_code()

    Performs a test of your code and prints the results
```

Project 5: Marketsim

`marketsim.py`

```
compute_portvals(orders_file='./orders/orders.csv', start_val=1000000, commission=9.95, impact=0.005)

    Computes the portfolio values.

    Parameters
        orders_file (str or file object) - Path of the order file or the file object
        start_val (int) - The starting value of the portfolio
        commission (float) - The fixed amount in dollars charged for each transaction (both entry and exit)
        impact (float) - The amount the price moves against the trader compared to the historical data at each transaction

    Returns
        the result (portvals) as a single-column dataframe indexed by date, containing the value of the portfolio for each trading day in the
        first column from start_date to end_date, inclusive.

Return type
    pandas.DataFrame


test_code()

    Helper function to test code


author()

    Returns
        The GT username of the student

    Return type
        str

study_group()

    Returns
        A comma separated string of GT_Name of each member of your study group
        # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

    Return type
        str
```

Project 6: Indicator Evaluation

`indicators.py`

Code implementing your indicators as functions that operate on DataFrames. There is no defined API for indicators.py, but when it runs, the main method should generate the charts that will illustrate your indicators in the report.

`marketsimcode.py`

This file is optional. It is an improved version of your marketsim code that accepts a “trades” DataFrame (instead of a file). Students can rearchitect their marketsimcode as necessary.

Note the file name change between Projects 5 and 6

`TheoreticallyOptimalStrategy.py`

```
testPolicy(symbol="AAPL", sd=dt.datetime(2010, 1, 1), ed=dt.datetime(2011,12,31), sv = 100000)

    Code implementing a TheoreticallyOptimalStrategy. It should implement testPolicy(), which returns a trades Pandas.DataFrame

    Parameters
        symbol    - the stock symbol to act on
        sd        - A DateTime object that represents the start date
        ed        - A DateTime object that represents the end date
        sv        - Start value of the portfolio

    Returns
        A single column data frame, indexed by date, whose values represent trades for each trading day
       (from the start date to the end date of a given period). Legal values are +1000.0 indicating a BUY
       of 1000 shares, -1000.0 indicating a SELL of 1000 shares, and 0.0 indicating NOTHING. Values of +2000
       and -2000 for trades are also legal so long as net holdings are constrained to -1000, 0, and 1000.
       Note: The format of this data frame differs from the one developed in a prior project.

    Return Type
        pandas.DataFrame


author()

    Returns
        The GT username of the student

    Return type
        str

study_group()

    Returns
        A comma separated string of GT_Name of each member of your study group
        # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

    Return type
        str
```

Project 7: Qlearning Robot

`QLearner.py`

```
class QLearner.QLearner(num_states=100, num_actions=4, alpha=0.2, gamma=0.9, rar=0.5, radr=0.99, dyna=0, verbose=False)

    This is a Q learner object.

    Parameters
        num_states (int)  - The number of states to consider.
        num_actions (int) - The number of actions available..
        alpha (float)     - The learning rate used in the update rule. Should range between 0.0 and 1.0 with 0.2 as a typical value.
        gamma (float)     - The discount rate used in the update rule. Should range between 0.0 and 1.0 with 0.9 as a typical value.
        rar (float)       - Random action rate: the probability of selecting a random action at each step. Should range between
                            0.0 (no random actions) to 1.0 (always random action) with 0.5 as a typical value.
        radr (float)      - Random action decay rate, after each update, rar = rar * radr. Ranges between 0.0 (immediate decay to 0)
                            and 1.0 (no decay). Typically 0.99.
        dyna (int) - The number of dyna updates for each regular update. When Dyna is used, 200 is a typical value.
        verbose (bool) - If “verbose” is True, your code can print out information for debugging.

    query(s_prime, r)

        Update the Q table and return an action

        Parameters
            s_prime (int) - The new state
            r (float) - The immediate reward

        Returns
            The selected action

        Return type
            int

    querysetstate(s)

        Update the state without updating the Q-table

        Parameters
            s (int) - The new state
```

```
        Returns
            The selected action

        Return type
            int

    author()

        Returns
            The GT username of the student

        Return type
            str

    study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str
```

testqlearner.py

```
# You do not need to modify this file. This is provided on the API Specificaion page for reference only
#

discretize(pos)

    convert the location to a single integer

    Parameters
        pos (int, int) – the position to discretize

    Returns
        the discretized position

    Return type
        int

getgoalpos(data)

    find where the goal is in the map

    Parameters
        data (array) – 2D array that stores the map

    Returns
        the position of the goal

    Return type
        tuple(int, int)

getrobotpos(data)

    Finds where the robot is in the map

    Parameters
        data (array) – 2D array that stores the map

    Returns
        the position of the robot

    Return type
        int, int

movebot(data, oldpos, a)

    move the robot and report reward

    Parameters
        data (array) – 2D array that stores the map
        oldpos (int, int) – old position of the robot
        a (int) – the action to take

    Returns
        the new position of the robot and the reward

    Return type
        tuple(int, int), int

printmap(data)

    Prints out the map

    Parameters
        data (array) – 2D array that stores the map

test(map, epochs, learner, verbose)

    function to test the code

    Parameters
        map (array) – 2D array that stores the map
        epochs (int) – each epoch involves one trip to the goal
        learner (QLearner) – the qlearner object
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output. When we test your code, verbose will be False.
```

```
    Returns
        the total reward

    Return type
        np.float64
```

Project 8: Strategy Evaluation

`StrategyLearner.py`

```
class StrategyLearner.StrategyLearner(verbose=False, impact=0.0, commission=0.0)

    A strategy learner that can learn a trading policy using the same indicators used in ManualStrategy.

    Parameters
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output.
        impact (float) – The market impact of each transaction, defaults to 0.0
        commission (float) – The commission amount charged, defaults to 0.0

    add_evidence(symbol='IBM', sd=datetime.datetime(2008, 1, 1, 0, 0), ed=datetime.datetime(2009, 1, 1, 0, 0), sv=100000)

        Trains your strategy learner over a given time frame.

        Parameters
            symbol (str) – The stock symbol to train on
            sd (datetime) – A datetime object that represents the start date, defaults to 1/1/2008
            ed (datetime) – A datetime object that represents the end date, defaults to 1/1/2009
            sv (int) – The starting value of the portfolio


    testPolicy(symbol='IBM', sd=datetime.datetime(2009, 1, 1, 0, 0), ed=datetime.datetime(2010, 1, 1, 0, 0), sv=100000)

        Tests your learner using data outside of the training data

        Parameters
            symbol (str) – The stock symbol that you trained on on
            sd (datetime) – A datetime object that represents the start date, defaults to 1/1/2009
            ed (datetime) – A datetime object that represents the end date, defaults to 1/1/2010
            sv (int) – The starting value of the portfolio
        Returns
            A single column data frame, indexed by date representing trades for each day. Legal values are +1000.0 indicating
            a BUY of 1000 shares, -1000.0 indicating a SELL of 1000 shares, and 0.0 indicating NOTHING.
            Values of +2000 and -2000 for trades are also legal when switching from long to short or short to
            long so long as net holdings are constrained to -1000, 0, and 1000.

        Return type
            pandas.DataFrame

    author()

        Returns
            The GT username of the student

        Return type
            str

    study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str
```

`ManualStrategy.py`

```
class ManualStrategy.ManualStrategy(verbose=False, impact=0.0, commission=0.0)

    A manual learner that can learn (essenitally human coded rules) a trading policy using the same indicators used in StrategyLearner=.

    Parameters
        verbose (bool) – If “verbose” is True, your code can print out information for debugging.
                         If verbose = False your code should not generate ANY output.
        impact (float) – The market impact of each transaction, defaults to 0.0
        commission (float) – The commission amount charged, defaults to 0.0

    add_evidence(symbol='IBM', sd=datetime.datetime(2008, 1, 1, 0, 0), ed=datetime.datetime(2009, 1, 1, 0, 0), sv=100000)

        # add evidence is used here for consistency with the Strategy Learner. In actuality, it is likely that
        # this function does nothnig as the "rules" will have been coded by the developer of the class. If that is the case,
        # this this method might simply consist of a "pass" statement.

        Trains your strategy learner over a given time frame.

        Parameters
            symbol (str) – The stock symbol to train on
            sd (datetime) – A datetime object that represents the start date, defaults to 1/1/2008
            ed (datetime) – A datetime object that represents the end date, defaults to 1/1/2009
            sv (int) – The starting value of the portfolio


    testPolicy(symbol='IBM', sd=datetime.datetime(2009, 1, 1, 0, 0), ed=datetime.datetime(2010, 1, 1, 0, 0), sv=100000)

        Tests your learner using data outside of the training data

        Parameters
```

```
            symbol (str) – The stock symbol that you trained on on
            sd (datetime) – A datetime object that represents the start date, defaults to 1/1/2009
            ed (datetime) – A datetime object that represents the end date, defaults to 1/1/2010
            sv (int) – The starting value of the portfolio
        Returns
            A single column data frame, indexed by date, representing trades for each day. Legal values are +1000.0 indicating
            a BUY of 1000 shares, -1000.0 indicating a SELL of 1000 shares, and 0.0 indicating NOTHING.
            Values of +2000 and -2000 for trades are also legal when switching from long to short or short to
            long so long as net holdings are constrained to -1000, 0, and 1000.

        Return type
            pandas.DataFrame

    author()

        Returns
            The GT username of the student

        Return type
            str

    study_group()

        Returns
            A comma separated string of GT_Name of each member of your study group
            # Example: "gburdell3, jdoe77, tbalch7" or "gburdell3" if a single individual working alone

        Return type
            str
```